

Xiang Li — Project Manual

Game systems, tools, and verification *What I am building, why it works the way it does, and how to check it.*

中文导读（给自己人看；发给面试官时删掉这一节即可）

这是第三版。相比上一版：**所有示意图都用英文重画，并且是照仓库里的真实配置数据画的**（9 节点路线图的节点与边、24 节点天赋树、结算四步与利息的例外处理、95/27 项校验），不再是中文旧图；Bell Garden 与 StylesVN 各自独立成章，配了系统层面的图与实机截图；FallenAngel 补了路线图、天赋树、结算管线、难度来源对比、验证清单、40 件 UI 资产总表。

- **面试官 / 外行（5 分钟）**：读每章开头的粗体句 + **In plain words** 段。
- **同行、设计师（15 分钟）**：读 FallenAngel 的 **Design decisions** 与 **What the numbers said**。
- **工程师（20 分钟）**：读 **Verification**，再看附录 A。

守望先锋 demo 是合作项目，不在本手册内。Bell Garden 与 StylesVN 均为个人项目（2026-09-17 确认）。配图在 `assets/`。MANUAL.html 是排版版，MANUAL.pdf 可直接附在邮件里。

How to read this manual

If you are...	Read...	Time
A recruiter, or someone who does not work in games	the bold line that opens each chapter, then the In plain words box	5 min
A designer or producer	Design decisions and What the numbers said in the FallenAngel chapter	15 min
An engineer or technical designer	the Verification sections, then Appendix A	20 min
Curious about the whole picture	read straight through	30 min

Who I am

I am a game designer working on **systems and the tooling around them** — the rules, the numbers, the tools other people use to author content with those rules, and the checks that tell me whether any of it works.

I am currently finishing an MSc in Business Administration (International Business) at the University of Amsterdam. Before that I worked in the industry: equipment and gem systems at SAGI Games, end-to-end production coordination and build testing at TAIKO Studios, and localization QA with cultural-risk review at MPMF. Every project in this manual is my own work.

Three threads

1. Design claims get tested, not asserted. If a design says "playing better should pay better", there is a simulation behind that sentence — and its boundary conditions are written down too.

2. Production problems get turned into tool problems. When content costs too much to produce, the answer is a tool, not more hours. When the tool is not good enough, that answer gets abandoned rather than defended.

3. Automation verifies state; people verify feel. Automated checks confirm that systems behave. Whether something *feels* right is decided by playing it. The projects keep those two separated on purpose.

On AI, since my CV says so directly: AI writes most of the implementation. The design decisions, the scope rules and the on-device testing are mine. The line I use is simple — **code implements decisions that have already been made; design is deciding what to make.** The FallenAngel chapter gives two examples of decisions that were made *against* more work.

The projects

#	Project	What it is	Status
01	FallenAngel	A rhythm game where songs are encounters inside a run, and you perform one instrument's part of each song	Framework and playing core working; content and the build layer next
02	Game Forge	A local dashboard that runs my projects and turns each one's README and Git state into the next three steps	In daily use; 11 automated tests
03	Bell Garden	A metroidvania vertical slice: abilities that open the map, enemies that read the player, a three-phase boss	Vertical slice playable; in progress
04	StylesVN	A visual-novel pipeline where the script is data: 9 scenes, 495 staged instructions	In progress
05	Study tools	A study engine with swappable course packs, and an exam platform built on it	In use

01 • FallenAngel

Rhythm games consume their own music: once you have FC'd or AP'd a song, you rarely come back to it. FallenAngel puts songs inside a run-based journey, so a song becomes a challenge you meet again instead of a trophy you retire.

In plain words. In most rhythm games you pick a song, chase a perfect score, and stop playing it once you finally nail it. FallenAngel does something different: you travel through a short map, meeting songs along the way, spending what you earn on your build, and going deeper — where the songs get harder and pay more. And you play one instrument's part of each song, so you hear lines in music you would normally never notice.

The problem: songs get collected and retired

Rhythm players have a word for finishing a song — 收曲, "collecting" it: you have full-comboed or all-perfected it. Once a song is collected, few players return to it. The song has become a trophy. It is the same structure as a boss you can only fight once, with the same result: the content is consumed by the act of mastering it.

That is not a difficulty problem, it is a design problem. And it has a second consequence: the only reason to keep playing is to get better at the game, which makes it a training tool for the people already invested, and an intimidating wall for everyone else.

The answer: a song is an encounter, not a trophy

Put the song inside a journey. In a run, you meet songs you have seen before — but meeting a song is not the purpose of the run. And because a run is not a practice session, you cannot guarantee the result you got in your best attempt. **Each encounter becomes a present challenge instead of a permanent one.**

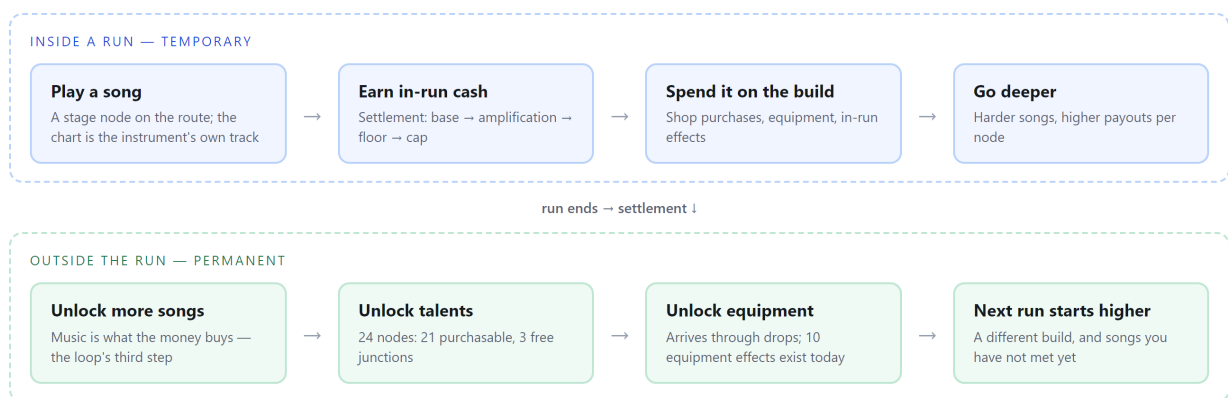
Three claims follow from that, and they are what make this game different from other rhythm games:

1. **You play for the music, not only to improve.** The roguelike frame gives a reason to play songs that is not "get better at this song".

2. **One song, many charts.** You are required to hit the instrument's own track — the bass line, the riff, the chords — so the parts that normally disappear under the melody become something you have to follow. The current implementation is attention-based: you hear the bass because you *have* to play it. The fuller version would be per-note, per-instrument sampling, which is shelved (see the status table).
3. **You cover one seat, not the whole song.** The other parts keep playing while you are responsible for one of them. For a player who does not read music, that is closer to the experience of being in a band — filling a part — than to performing a piece. The name for it internally is a *musician's experience card*.

Two loops, one settlement

A single run feeds permanent progression; permanent progression changes the next run.



The design claim this diagram carries: the run is where the challenge lives, and the permanent layer is what makes the next run a different challenge rather than a repeat of the last one.

Two loops, one settlement: a run feeds the permanent layer, and the permanent layer changes the next run

Why it is not just another progression machine

Both structures have growth, so the difference has to be somewhere else — and it is in **where the difficulty comes from**.

Where the difficulty comes from

Both games have progression. The difference is who is expected to change.

Traditional rhythm game

Difficulty is **fixed** — the chart does not change.

The **player** grows until the fixed chart is beaten.

Goal: grind one chart to perfection.

Songs are consumed once cleared: **collected and retired**.

FallenAngel

Difficulty **rises with depth** along the run.

The **build** is what offsets it — money, equipment, talents.

Goal: carry a build up a steeper curve.

Songs return as **present challenges**, never as permanent ones.

This is also the honest boundary of the current build: the run structure and the out-of-run framework are demoable, while the in-run build layer is still a design problem rather than a production one.

Difficulty source: a fixed chart the player grows into, versus a curve the build offsets

The goal inside a single run, in my own words:

Pass nodes safely and accumulate in-run currency to develop your build, then go deeper. Songs deeper in the run are harder, while buffs and equipment push the difficulty back down. Deeper nodes pay more, and at the end of the run everything settles into out-of-run resources.

The core loop is: **play** → **earn** → **unlock more songs / talents / equipment** → **play (more songs)** → **earn (more)** → **unlock more**. Music's place in that loop is the third step: money buys access to more music.

What a run looks like

The route is deliberately short — nine nodes — so every branch is a legible choice rather than a maze. This is the actual configuration, node by node:

The route: 9 nodes, one paid fork

Node types and edges as they exist in the configuration: N00–N08, with the only paid edge at fare 40.



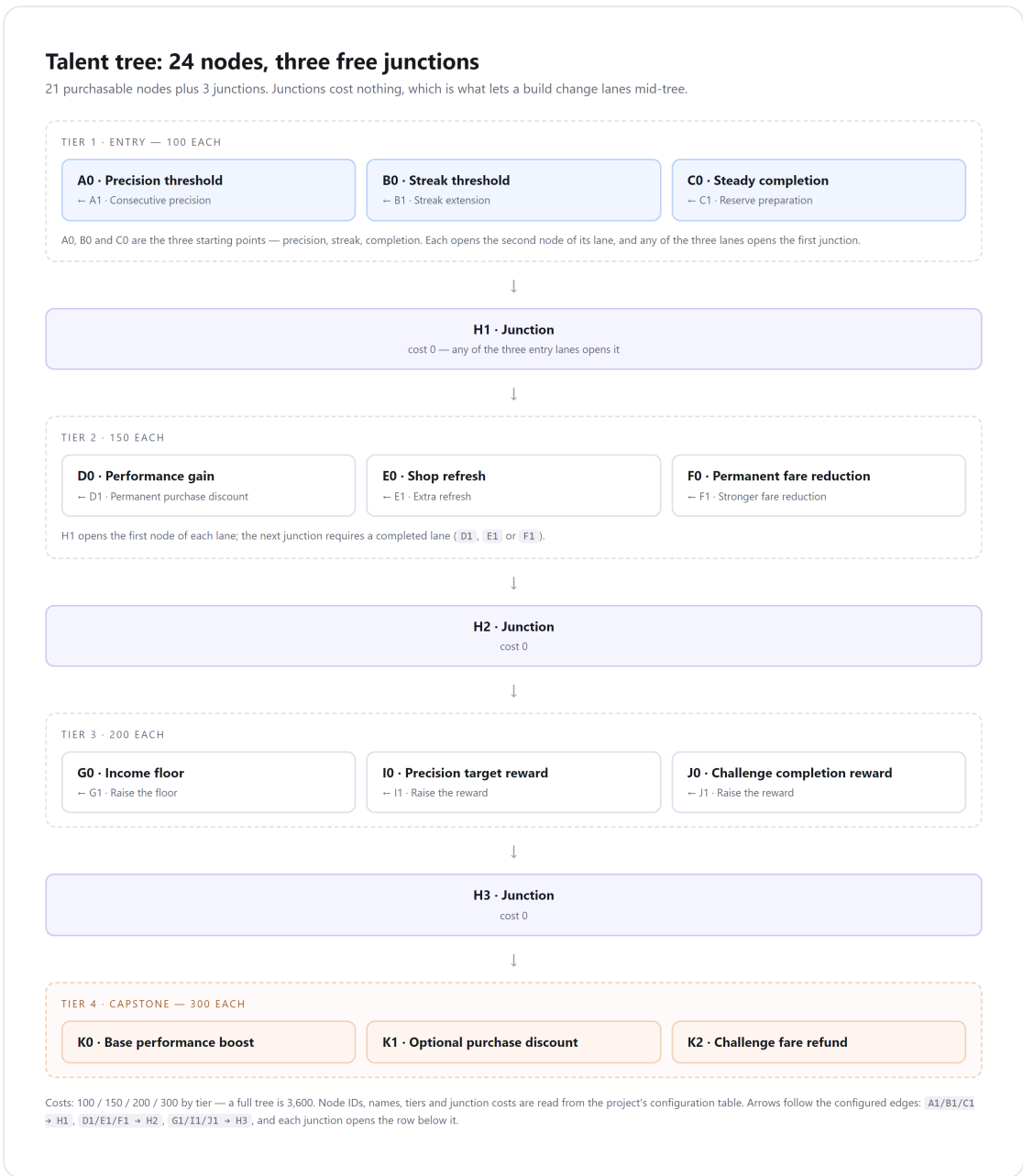
The icons are the real UI assets shipped from `Assets/Art/ui`, drawn from the same geometry definition the game renders with. The fare was tuned from 60 to 40 during balance work.

The route: START, a stage, a shop, a fork with one paid edge, then a final stage

There are four node types — stage, shop, empty and final — and exactly **one paid edge**, priced at 40 after balance tuning. The paid branch leads to a harder song (S02) instead of the repeat (S01); the free branch is the safe route. The room icons in the diagram are the shipped UI assets, not a mock-up.

The permanent layer

Twenty-four talent nodes, of which twenty-one are purchasable and three are free junctions. The junctions are the important part: because they cost nothing and accept any of the lanes feeding them, a build can change direction mid-tree instead of being locked into the choice made at level one.



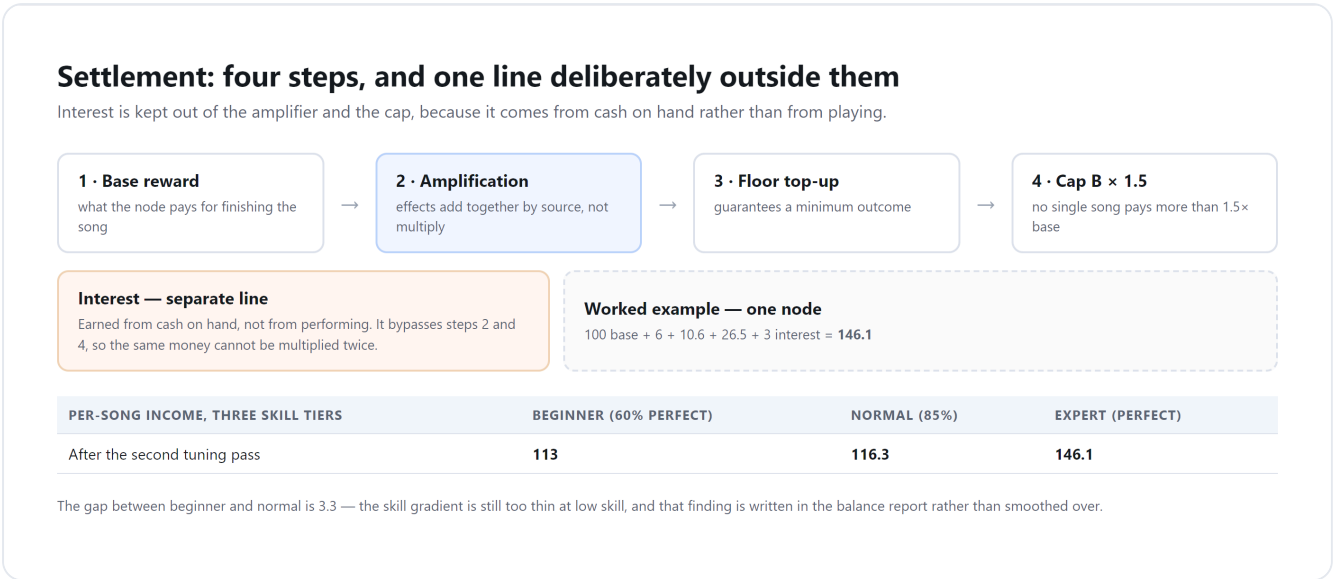
The talent tree: three lanes, three free junctions, four cost tiers

Costs run 100 / 150 / 200 / 300 by tier, so a full tree is 3,600. Node names, tiers and costs in the figure come from the project's configuration table.

Design decisions

1. Charts follow the instrument's own track, not the melody. The obvious charting choice is to follow the melody or the drums. The rule here is: whichever instrument the part belongs to, that is the track you hit. *The trade-off:* charts are harder to author and less immediately readable to a newcomer — but it is what makes claim #2 real rather than decorative. Without it, "you hear the bass" is a promise the game does not keep.

2. Interest is deliberately kept out of the settlement amplifier.



Settlement: base → amplification → floor → cap, with interest handled as a separate line

Interest comes from cash on hand, not from playing, so if it entered the amplifier and the cap, the same money would be multiplied twice. *The cost:* a longer pipeline to explain, and it introduces cash-hoarding builds that will need balancing later.

3. Tools instead of manpower — and a willingness to stop. The bottleneck is content: song library, charts, and multi-part chart instances. The answer is tooling. The chart editor handles 5 tracks with waveform alignment, editing while the audio plays, box and multi-select, copy/paste, cross-track dragging, 60-step undo, live validity checks and import of open chart formats. Auto-conversion from real scores was also tried — and **stopped**: the output was not good enough on any instrument track, and reviewing generated charts cost more than authoring them. Stopping was the cheaper decision and I made it explicitly.

4. Whether judgement feel should change is an experience question, not a preset. I am not committing in advance to widening or moving the judgement windows; the decision belongs to hands-on play. But the answer splits cleanly in two: if windows become purchasable, then "all perfect" under a widened window is no longer the same achievement, and technical expression gets diluted — that part can be reasoned about up front. Whether it *feels* good, and whether players feel they are being sold power instead of skill, has to be tested.

What the numbers said

I built a balance simulator rather than trusting my own sense of the economy: 15 runs × 5 seeds × 3 skill tiers (a perfect run, 85% perfect, 60% perfect), with a greedy strategy — buy the cheapest affordable equipment, unlock the cheapest available talent. It reads the same JSON the game exports and runs the same loop (route → playing income → shop → talent unlock).

The report opens with a section titled **model assumptions**, listing what is *not* modelled: failure, interruption, real purchase decisions, and player-to-player variance. That section is the one I would point at first, because a simulation is only useful if its boundary is stated.

The result after the second tuning pass — tiered thresholds on one effect, fare 60 → 40, one item 80 → 60 — was per-song income of **113 / 116.3 / 146.1** across the three skill tiers. The honest part: beginner and normal are **3.3 apart**. The skill gradient at low skill levels is still too thin, and that is written in the report rather than smoothed over. A worked example from the settlement pipeline, for one node: 100 (base) + 6 + 10.6 + 26.5 + 3 (interest) = **146.1**.

Status, layer by layer

I would rather present this as a table than as a claim:

Layer	State
Run structure (the route)	Demoable — 9 nodes, four node types, one paid fork at fare 40
Out-of-run build framework	Demoable
Playing core	Being polished — 4-key judgement core runs; windows aligned to Phigros (PERFECT ± 80 / GREAT ± 160 / GOOD ± 180 / BAD ± 200 ms); beat calibration; hold, slide, wide-key and five note types
Playable charts	Two internal test charts (1054 and 1368 notes) converted from real scores — no licence, so internal testing only
Multi-part charts (one song, many charts)	Format supports 4-key drum and 5-key guitar layouts; only the drum path runs today , and there is no externally visible instance yet
Equipment drops	Implemented — weighted draw, exclude already-owned, empty pool pays nothing, failure triggers nothing; 5 related checks green
Songs as drops	Not wired in — the drop table contains equipment and placeholder items; songs are not among them
Build design (how money reduces risk and adds leverage)	Designed in scope, not yet designed in detail — a design problem, not a production problem
Risk and modifier space (purge, wider judgement, revive, one-use instrument-swap card)	Not implemented. The instrument-swap card is the piece that would give multi-part charts a mechanical reason: mid-run you may be forced onto a different instrument's chart
Auto chart conversion	Stopped — quality was not good enough on any instrument track, and reviewing generated charts cost more than authoring them
Per-note, per-instrument sampling	Shelved — the original ambition was to sample every note of every instrument track so each hit is audible; not finishable at solo-developer capacity

Verification

What is actually checked

Automation verifies state. People verify feel — the two are kept apart on purpose.

95 checks

Batch-mode run over configuration and rules; recorded PASS with 0 compile errors

27 geometry checks

Icon ink bounding-box centre against the host rectangle, plus overflow

Automated UI checks

Clicking the real interface — its own header states it is *not* a human play test

Transactional save

Temp file → replace, revision compare-and-swap; a failed write leaves no side effects

730-note round-trip

Chart → MusicXML → chart, zero differences

Balance simulation

15 runs × 5 seeds × 3 skill tiers, with a written list of what is *not* modelled

7 art workflows

ComfyUI graphs used to fix direction, not to produce final assets

40 delivered PNGs

UI exports drawn from the same geometry definition as the in-game art

Two playable charts exist for internal testing only — 1054 and 1368 notes — because the music is not licensed.

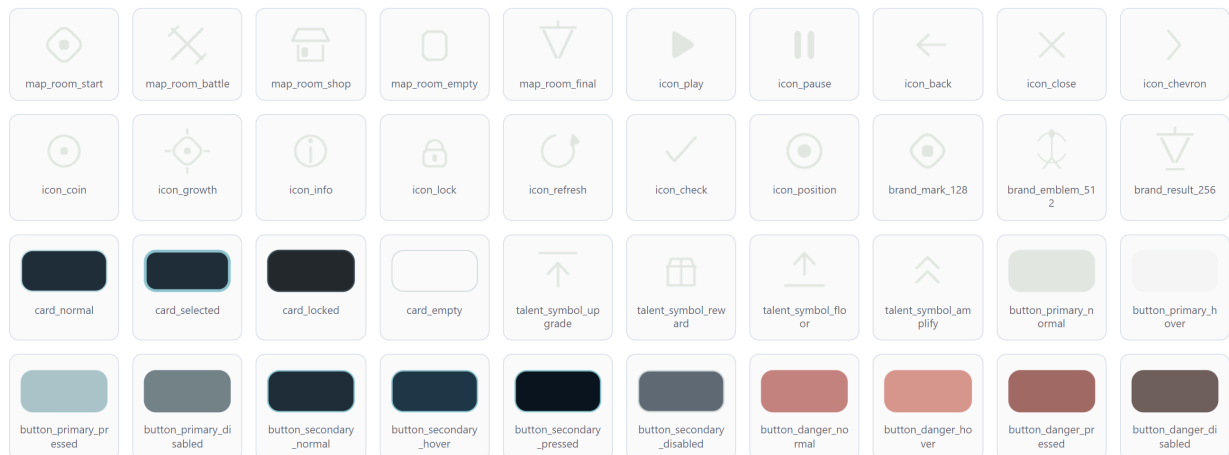
What is checked: batch-mode checks, geometry checks, transactional saves, the simulation, and the art pipeline

- **95 checks** run in Unity batch mode; the recorded run on 2026-09-10 was PASS 95 with 0 compile errors, logged to file.
- **27 icon geometry checks** — does the ink bounding box centre match the host rectangle centre, and does anything overflow its frame.
- **A separate automated UI check class** drives boundary cases by clicking the real interface — and its own header comment states that it is *not* a human rhythm-play test.
- **Transactional saves** — write to a temporary file, then `File.Replace`, with a revision compare-and-swap.
- **Chart format round-trip** — chart → MusicXML → chart, zero differences on a 730-note case.

The art pipeline follows the same rule as the rest of the project: produce direction quickly, then deliver assets that cannot drift apart.

40 delivered UI assets from one geometry definition

Buttons in four states, cards in four states, icons, and the room symbols the route map is drawn from.



Because the exported files and the in-game drawing share the same geometry, a change to one cannot silently leave the other behind.

40 delivered UI assets drawn from one shared geometry definition

The question the CV invites

"Your CV says AI wrote most of the code. So what did you do?"

The answer is one sentence: **the code implements decisions that were already made; the design is deciding what to make.** A producer's job includes pricing the work — not spending tokens on lottery tickets, and not handing "invent a game" to a model that can only recombine.

Two examples where the decision was *not* to build more:

- Auto chart conversion was stopped when the output was worse than hand-authoring on every instrument track.
- Per-note, per-instrument sampling was shelved when the cost exceeded what one person could finish.

What not to assume

- There is no content library yet. The two playable charts are internal test material without a licence.
- The in-run build layer is scoped but not designed in detail.
- Modifiers, the instrument-swap card, and songs as drops are proposals, not features.

- Multi-part charts have no externally visible instance yet, and per-note sampling is shelved.

Everything above is written down in the repository's roadmap, split into design (8 items), content production (5), engineering and verification (8) and pre-release (4), with dependencies and a suggested order.

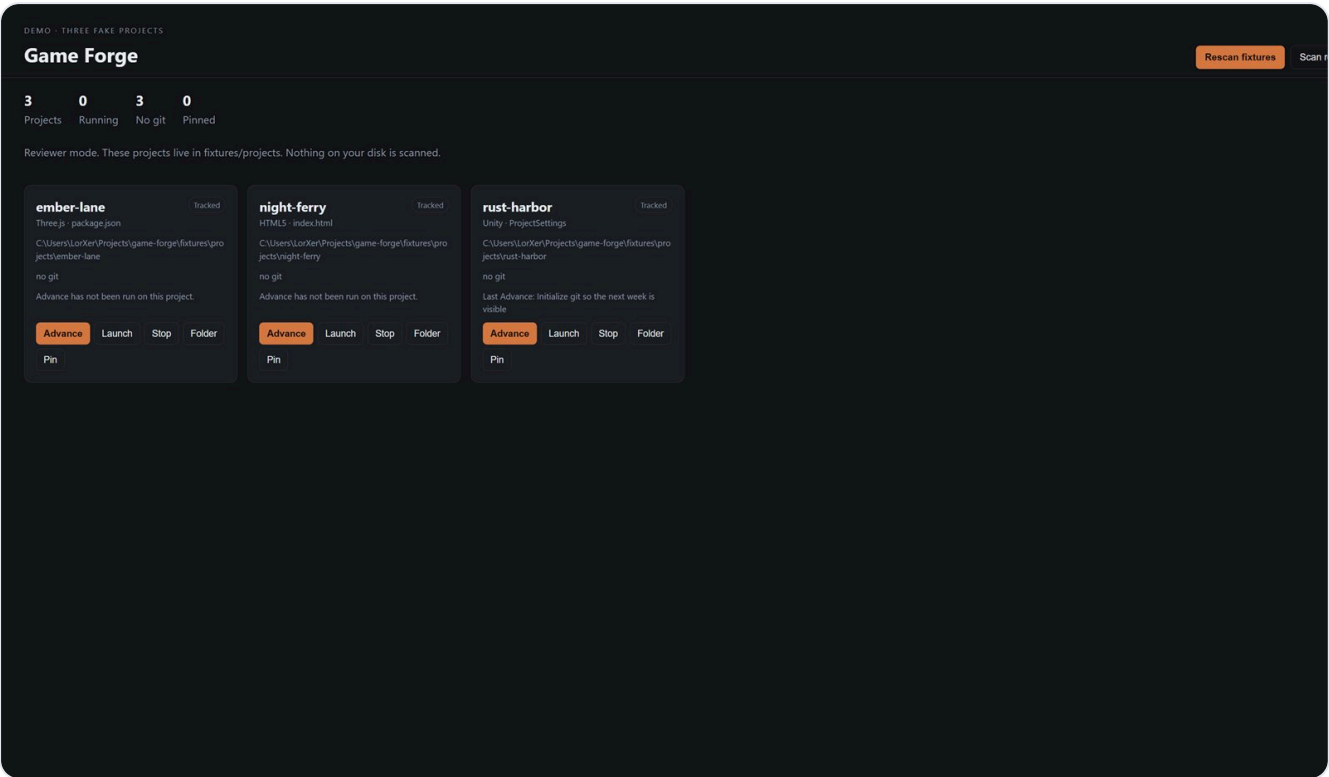
Where to look. `README.md` (bilingual), `docs/architecture.md`, `docs/chart-editor.md`, `docs/chart-format.md`, `docs/roadmap.md`, `portfolio/validation_summary.md`, and the balance report. Repository: `github.com/lixiang021011-sketch/FallenAngel` (tag `v0.3.0`).

02 • Game Forge — the dashboard I built to run my own projects

A local tool that scans the machine for game projects, launches any of them, and turns each project's README, tasks and Git state into the next three actionable steps.

In plain words. When your work is spread across six projects on four drives, starting a session costs ten minutes of "what state is everything in, and what was I doing?". Game Forge answers that in one screen: every project, whether it is running, what its Git state is, and what the next useful step would be. It is the same instinct as the chart editor — a production problem answered with a tool.

At a glance	
What it is	Zero-dependency Node.js tool with a browser dashboard
Scope	Scans for Unity, Godot, Unreal, Three.js, HTML5 and runnable Node projects
Runs	Locally only — bound to <code>127.0.0.1</code> , nothing uploaded
Quality	11 automated tests; a demo mode that runs on fixtures
Status	In daily use



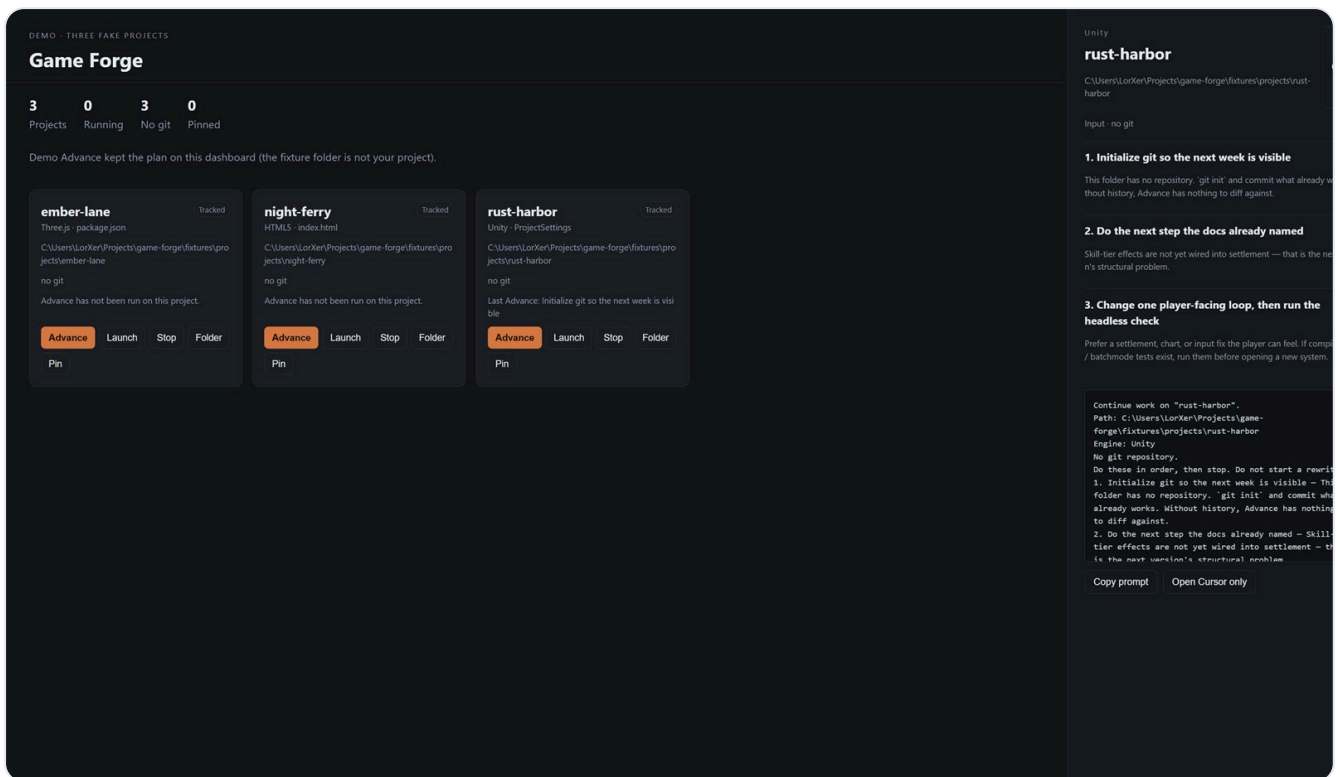
The dashboard in demo mode: projects, runtime state and Git state in one view

Design decisions

1. **Detect by markers, and explicitly distrust vendor files.** Projects are identified by real signals — `ProjectSettings/` , `project.godot` , a `.uproject` , a runnable `package.json` , or a `.game-forge/project.json` marker of my own. This came out of a real failure: a leftover bundle directory was being registered as a project because its README belonged to a bundler. The fix is in the detector and is covered by a test.
2. **Zero dependencies.** A personal tool should still work in three years: no install step, no version rot, nothing to audit.

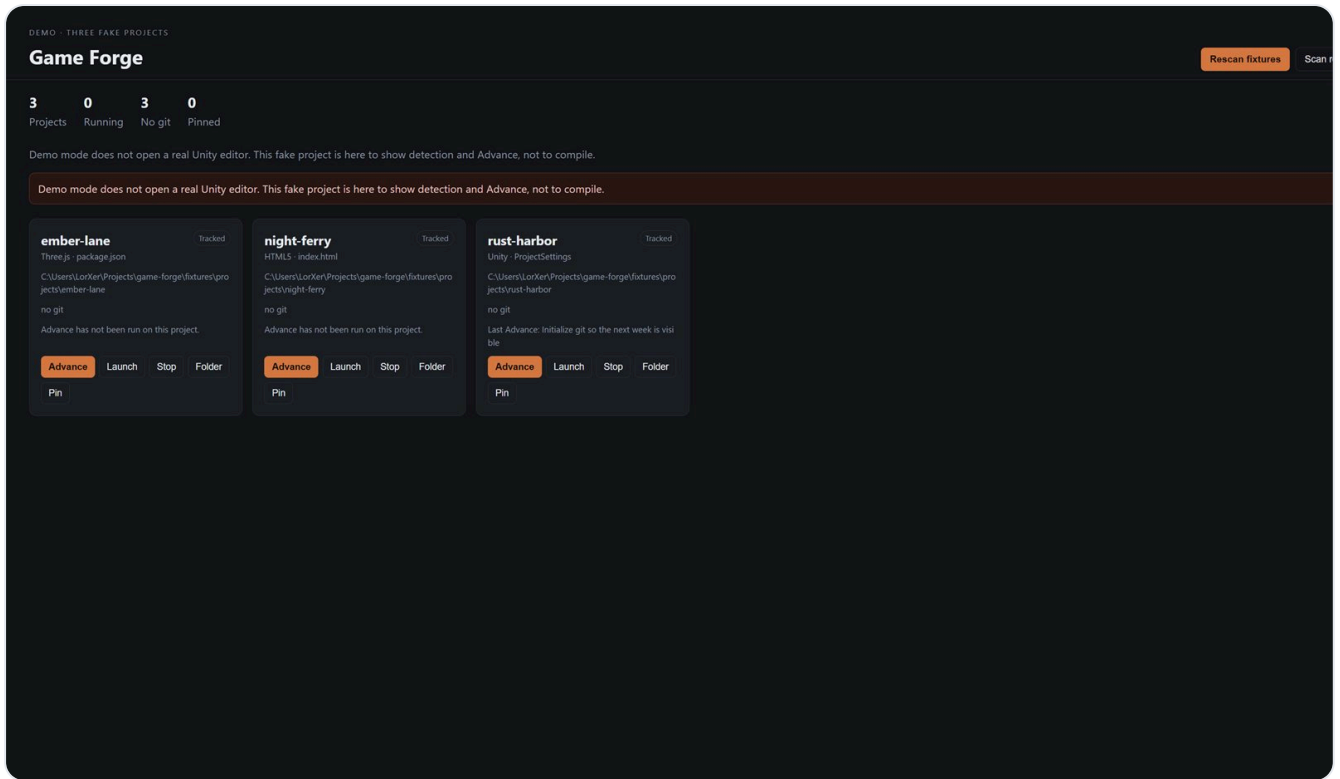
3. **Deliberately small side effects.** One file written per action, no project code touched, no commits made on my behalf.
4. **One work, one entry.** Two copies of the same game collapse into a single card, so the dashboard shows what I am working on rather than where files happen to live.

What was hard. The interesting half is "Advance" — turning a project's README, TODOs and Git state into three sensible next steps. To show it is not decoration, `docs/evidence/fallen-angel1-advance.md` runs the *same* project through three input states — a dirty working tree, a clean tree, and a project without Git — and records how the suggestions change.



Advance on a project with no Git: three next steps plus the prompt that produced them

The failure path is part of the design too: when a project cannot be launched, the tool says why instead of pretending it worked.



Launch failure surfaced in the interface rather than a silent no-op

Verification

- `npm test` — 11 tests over detection, launching and advance generation, including regressions for the two failures above.
- `npm run demo` — a harbor built from fixtures, so a reviewer can run the interface without granting access to their disk.
- `npm run evidence` — regenerates the three-state Advance evidence.

Where to look. `README.md` (Chinese) and `README.en.md` (English); `docs/screenshots/` for the walkthrough, `docs/evidence/` for the Advance contrasts, `src/` for the detector, launcher and planner.

03 · Bell Garden — a metroidvania vertical slice

Nine areas, three abilities and a three-phase boss, in a slice that plays itself to

completion as a regression check.

In plain words. It is a small but complete piece of a metroidvania: you explore connected areas, find abilities that open places you could not reach before, and fight enemies that react to what you do. The part worth discussing is not the slice itself — it is that I audited the first version, compared it against open-source examples of the genre, and then rewrote the systems and the art instead of patching a structure I no longer trusted.

At a glance	
What it is	Vertical slice: 9 areas / 15 rooms, 3 movement abilities, 4 enemies, one three-phase boss
Stack and platform	Unity; Windows and Android builds
My role	Level and combat design, systems, art direction, automated regression
Status	In progress



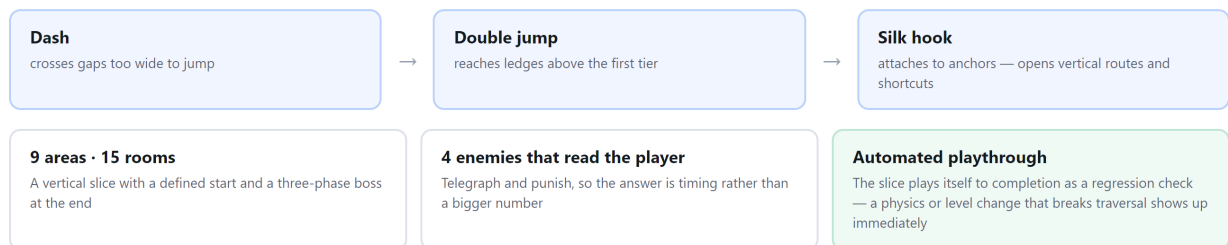
Bell Garden title screen

Design decisions

1. **Abilities gate classes of space, not single doors.** Dash, double jump and silk hook each open a *kind* of traversal, so the map communicates progress by itself — you do not need a locked door to tell you that you are not ready yet.
2. **Enemies read the player instead of absorbing damage.** All four enemy types telegraph and punish, so the answer to a fight is timing rather than a bigger number. *The trade-off:* much harder to balance, and it makes a three-phase boss the natural endpoint of a slice this size.
3. **Rewrite rather than patch.** After the audit I kept the ideas that held up and rebuilt the systems and the art. A rewrite is expensive, but patching a structure I no longer trusted would have cost more later.

Bell Garden: abilities as the map's grammar

Each ability opens a class of space rather than a single door, so progress is read from the map itself.



In progress. The rewrite decision — auditing v1 against open-source examples and rebuilding rather than patching — is the part of this project worth discussing.

Ability gating: each ability opens a class of traversal, and the slice checks itself by playing to the end

What it looks like



An area in progress — combat and traversal read from one screen at a time



The map view: a first-class feature, because a 15-room map has to stay legible



The three-phase boss encounter

What was hard. Making a fifteen-room map readable when the player only ever sees one screen at a time. That is why the map became a first-class feature rather than a debug overlay.

Verification. The slice ships with an automated run that plays it from start to finish and reports the outcome, so a level or physics change that breaks traversal is visible immediately.

Where to look. `README.md` in the project, plus the separate audit-and-rewrite write-up that compares the first version against open-source references point by point.

04 · StylesVN — a narrative pipeline in Unity

A full-length visual novel built to test one idea: that a script can drive everything. Nine scenes, 495 staged instructions, 437 lines of dialogue.

In plain words. A visual novel is an interactive story — text, portraits, backgrounds and sound, assembled into scenes the player clicks through. Writing one is a content problem, and

content problems are solved with pipelines. I built the pipeline, then loaded it with a complete story so that it had to survive real weight.

At a glance	
What it is	Nine chapters, ~22,600 characters of dialogue, ~100 minutes of play
Assets	13 scene backgrounds · 13 character portraits · title art
Stack and platform	Unity 2022.3 LTS, built-in render pipeline; Windows and Android
My role	Script adaptation, scene scripting, tooling, art direction, licensing
Status	In progress



StylesVN title art

Design decisions

1. The script is data, not code. Every scene is a list of instructions — background, who speaks, which portrait, which sound, which transition. Adding a scene means adding data, not touching the engine. That is what makes a script of this length maintainable by one person.

StylesVN: the script is data, not code

A scene is a list of instructions. Adding a scene means adding data — the engine does not change.

content/

chapters/ch01–ch09.json — 9 chapters of staged instructions
characters.json · evidence.json · investigations.json
interrogations.json · accusation.json · stats.json
index.json — what the loader reads first

Scale

495 staged instructions · 437 lines of dialogue · ~22,600 characters · ~100 minutes

Assets

13 scene backgrounds · 13 character portraits · title art

```
// an excerpt from chapters/ch01.json — the real instruction schema
{ "t": "bg", "id": "styles_manor_dusk" },
{ "t": "say", "who": "narr",
  "text": "...", "figs": [] }
```

Every beat carries its own type — background, speech, portrait, sound, transition — so the scene file is the single source of truth for what the player sees and hears.

The content pipeline: chapter files plus shared data, and the instruction schema they share

2. A two-stage check instead of a rebuild. Script errors used to surface only after a scene was rebuilt. Now one command compiles, validates the content, and then plays the entire story through to its ending — turning a two-minute feedback loop into a two-second one.

StylesVN: two-stage check, one command

Script errors used to surface only after a scene rebuild. This turns a two-minute loop into a two-second one.

check.cmd

run from the project root

1 · Compile + content validation

Batch mode; compiler errors and content problems both fail the run, with a report written to ToolsOut/

2 · Full-story self test

Plays the whole script through to the ending automatically

3 · Player-side health check

32 checks inside the built executable

The same idea appears in every project here: make the feedback loop small enough that correctness is cheap. It is the reason the script could reach 495 instructions without becoming unmaintainable.

One command, two stages, plus a player-side health check inside the build

3. Licensing handled up front, not at release. Code is MIT; story and art are all-rights-reserved with an explicit fan-work clause; every third-party font and package is itemised in a notices file. This matters because the project is an unofficial adaptation — I would rather state the boundary precisely than leave it implied.

What it looks like



A scene background from the project



One of thirteen character portraits

What was hard. Holding a long-form script debuggable. At 495 instructions, a typo and a design mistake look identical in the output; the checker, the instruction schema and consistent naming are what make the difference visible.

Note on the source material. StylesVN is a non-commercial, unofficial adaptation of Agatha Christie's *The Mysterious Affair at Styles*. The Chinese script is my own rewrite; no published translation was used; the project is unaffiliated with the rights holders and will be taken down on request. I chose an existing story deliberately: it gave me a real narrative workload so I could build and test the pipeline, instead of spending the same time inventing 22,000 characters of plot.

Where to look. Repository github.com/lixiang021011-sketch/StylesVN — source, script and art, with builds kept out of the repository and licensing split across code, assets and third-party notices.

05 • Study tools

The same move as Game Forge and the chart editor, applied to learning: keep the engine, swap the content.

Study Kit is a study application built around one decision: the engine — question practice, mock exams, wrong-answer book, notes, PDF reading — is separate from the course packs. Changing subject changes the content; the engine stays. A second project applies the same engine to one specific certification, reading textbook PDFs and a question bank, with chapter and question-type filters, timed mock papers, and source-page jumps from a question back into the textbook.

They are here because they are the cleanest example of a move that appears in every project above: **separate the part that stays from the part that changes**. In Game Forge that is the scanner versus the project list; in StylesVN it is the instruction schema versus the chapters; in FallenAngel it is the chart format versus the chart content. Both tools are private and in personal use.

Appendix A • Verify it yourself in 10 minutes

I would rather be checked than believed.

FallenAngel (public repository)

```
git clone https://github.com/lixiang021011-sketch/FallenAngel
cd FallenAngel
git log --oneline | wc -l      # 75 commits, tagged v0.3.0
```

- `README.md` , first screen → the loop and the economy figures.
- `Assets/Scripts/Data/PortfolioConfig.cs` → the map nodes and edges, the 24 talent nodes and their costs, and the 31 effects that the diagrams in this manual were drawn from.
- `docs/architecture.md` , `docs/chart-editor.md` , `docs/chart-format.md` → how the systems and the editor work.
- `docs/roadmap.md` → what is designed, what is queued, and in what order.
- `portfolio/validation_summary.md` and the balance report → the 95 checks, the 27 icon checks, and the simulation with its stated model assumptions.

StylesVN (public repository)

```
git clone https://github.com/lixiang021011-sketch/StylesVN
```

- `Assets/StreamingAssets/content/chapters/ch01.json` → the instruction schema the pipeline is built on.
- `check.cmd` → the two-stage check, including the full-story self test.
- `LICENSE-CODE` , `LICENSE-ASSETS` , `THIRD_PARTY_NOTICES.md` → how the licensing is split.

Game Forge (available on request)

```
npm test      # 11 tests: detection, launching, advance generation
npm run demo  # full interface on fixtures – your disk is never scanned
```

- `docs/evidence/fallen-angel-advance.md` → one project under three input states, with the three next steps each produces.
- `git log` → the two real defects that are now regression tests.

A standing offer. Ask me about any decision in this manual — including the ones I got wrong, stopped, or have not designed yet — and I will walk you through the trade-off.

Appendix B • Glossary for non-specialist readers

Term	What it means here
FC / AP	"Full combo" and "all perfect": clearing a song without breaking the combo, or hitting every note at the top judgement.
收曲	Rhythm-game slang for finishing a song for good. Once collected, most players never return to it — the problem this project is built around.
Chart	The file describing when notes appear in a song and which key plays them.
One song, many charts	Splitting a song into instrument parts so the player performs one part — the bass, the guitar — instead of the whole mix.
Build	The set of upgrades, equipment and effects a player assembles, and what they choose to spend money on.
Roguelike / run	A structure where each attempt is temporary, and choices made inside it change what you carry forward.
Settlement	The moment after a run when earnings convert into permanent progress.
Talent tree / junction	A map of permanent upgrades; a junction is a free node that lets a build change lanes instead of locking in an early choice.
Difficulty source	Where the challenge comes from: a fixed chart the player grows into, or a curve that rises and a build that offsets it.
Systems design	Designing the rules and their relationships — what produces what, and what limits it.
Numeric design / balance	Choosing the numbers so the rules produce the intended experience, usually with a model rather than by feel.
Config table	A spreadsheet-like file of values that can be tuned without touching code.
Judgement window	The timing tolerance for each accuracy grade — how early or late a hit still counts as perfect.
Vertical slice	A small but complete piece of a game, built to prove the whole works — not a demo of one feature.

Term	What it means here
Metroidvania	An exploration game where new abilities open previously unreachable parts of a connected map.
Visual novel	A story-driven game of text, portraits and backgrounds, where the player advances scenes.
Headless check	An automated test that runs without opening the game, fast enough to run before every save.
Regression test	A test written after a bug, so the same bug cannot return unnoticed.
Transactional save	A save that either completes fully or leaves the previous state intact — never a half-written file.
Commit / tag	A saved checkpoint in a project's history; a tag is a named checkpoint such as <code>v0.3.0</code> .

Appendix C • Project index

Project	One line	Status	Where to look
FallenAngel	Rhythm game where songs are encounters in a run, and you perform one instrument's part	Framework and playing core working; content and build layer next	Public repository, tag <code>v0.3.0</code>
Game Forge	Local dashboard that runs my projects and generates the next three steps for each	In daily use	Available on request
Bell Garden	Metroidvania vertical slice: 9 areas, 3 abilities, a three-phase boss	In progress	Available on request
StylesVN	Visual-novel pipeline: 9 scenes, 495 instructions, ~22,600 characters	In progress	Public repository
Study Kit	Study engine with swappable course packs	In use	Private
Exam platform	Applies the engine to one certification, with PDF source-page jumps	In use	Private

Contact

Xiang Li — Amsterdam, the Netherlands `lixiang021011@gmail.com` · +31 6 17135117 · `github.com/lixiang021011-sketch`

Happy to demo the run loop, walk through the balance model, or talk about any decision I stopped or shelved.